

ENGINEERING DESIGN PAPER

Lamina: an inspectable runtime for document production

Fadi Bahodi · September 2026

Lamina is an open-source system for producing documents from changing source collections. A model still does the reading and writing. The system controls which source material each model call receives, how independent work is scheduled, which results can be reused, and what evidence a reviewer can inspect. Its first working application produces source-linked learning guides. A separate, newer runtime executes reusable job graphs. They share infrastructure but are not yet one integrated production path.

The customer problem is familiar to teams that maintain training material, operating guides, policy manuals, or technical briefs. A new source arrives; an old statement is corrected; the intended audience changes. Regenerating everything is slow and can introduce fresh errors. Editing the final PDF alone may leave the reasoning and source trail inconsistent. A useful system should tell an editor what source changed, which decisions used it, what needs reconsideration, and which completed work can remain. Lamina currently solves pieces of that problem. This paper sets out an architecture for connecting them and names the evidence needed before claiming it works better than existing products.

The central design choice is to treat document production as a series of decisions about source, meaning, form, work allocation, and delivery. A job graph coordinates those decisions; it does not make them correct by itself. The proposed system needs both a reliable execution substrate and a representation of what the workers are trying to preserve or change.

1. The production task

A document request begins with a task contract. It names the intended reader, the result to be delivered, the source collection, the deadline or cost constraint, and the standard for acceptance. “Turn these files into a guide” leaves critical decisions implicit. One guide may need every policy exception; another may need a short operating route. A reference manual and a spoken lesson impose different demands on source coverage and wording. The contract should specify who may decide what to omit and how that decision will be reviewed.

This specificity serves a business purpose. Teams can reuse a production method only if a later operator can tell whether it fits the new task. A method for an internal incident guide may be appropriate for a small, stable source collection. It may fail on a long regulatory manual with diagrams, conflicting revisions, and a strict publication gate. Reuse without applicability conditions gives a faster route to the wrong artifact.

Lamina's current guide builder accepts files and a bounded teaching procedure. The procedure sets audience, instructions, outputs, and worker count. It does not choose a new execution graph. The method runner accepts a separate method file and task JSON. It can execute that graph through an operator-configured adapter, but it does not automatically parse the sources or inherit the guide builder's source checks. The intended architecture connects those paths through an explicit task contract and typed stage results. That integration is proposed, not present in the released guide workflow.

2. Working code and open design

The present implementation is useful as a base, and its limits are precise. The table separates what runs today from mechanisms that are partly present or still need implementation.

Capability	Status	Evidence in this repository
Stable file identity, ordered text units, and source locations	Implemented for Markdown, plain text, and extractable PDF text	src/lamina/ingest.py
Extract, reconcile, plan, author, review, and export a source-linked guide	Implemented as a fixed pipeline	src/lamina/pipeline.py, src/lamina/validation.py, src/lamina/export.py
Shared route and exact prerequisite source evidence for parallel guide authors	Implemented, with planned context rather than finished earlier prose	src/lamina/pipeline.py
User-defined graph of bounded JSON jobs, resource lanes, dependency results, local cache, and run receipts	Implemented in a separate runtime with CLI and local browser access	src/lamina/methods.py, src/lamina/method_runtime.py
Operator-selected past observations in a later node's context	Implemented for the separate method runtime	src/lamina/method_runtime.py
Task-specific visual source inventory, method selection, typed semantic dependencies, and finding-directed repair	Proposed	No connected implementation in the current guide or method path
Verified external delivery and observed reader use	Proposed	Current guide exports files and a static reader; Listen is a script

The guide's exact-quote check establishes that a quoted string occurs in an allowed source unit. It does not establish that the author's sentence follows from it. The plan accounts for concepts the extractor returned; it cannot prove the extractor found every important concept. PDF text

extraction can lose a diagram or table relation before a model ever sees it. Generic method nodes have an even narrower validator: they must return JSON objects, and the adapter or a later job must judge their domain meaning.

These distinctions matter when communicating maturity. The current browser app can show a curated example and run the fixed guide builder locally through a configured adapter. The custom method graph runs from the command line or the local browser workbench, which imports a method and task, validates the graph, starts execution, and displays the resulting receipt. A stored observation can be deliberately selected for a later method node; no program chooses that observation, revises the method, or claims to have learned a better one automatically. None of the current demos establishes a quality or speed advantage on unseen customer material.

3. Source policy before extraction

The intended task contract needs a source policy that says how each input may be used. An authoritative procedure, an older contradictory note, an example of desired style, and a held-out assessment item should not all enter the same prompt. The current guide pipeline distinguishes teaching files from held-out assessment files. That is a sound starting boundary, but many production jobs need a richer policy. A style example can inform tone without supplying factual claims. A superseded policy may be evidence of what changed without being current authority. A reviewer may need to see a conflict that an author should not resolve alone.

The physical source packet also matters. If a chart is cropped, a model cannot reconstruct the missing axis from an instruction to “focus on the chart.” If a table's reading order is broken, exact quoted words may support a false relationship. A source-region inventory should identify prose, tables, figures, captions, formulas, scanned pages, and links between them. Each region needs a locator, content fingerprint, parse status, and a way to reopen the original. Unsupported regions should be visible as unsupported. A processed-file count must not be presented as understood-source coverage.

The first integration should not assume one universal parser. A short Markdown brief can use its headings and paragraphs. A figure-heavy PDF may need page rendering, image extraction, or manual region selection. The method should declare which source classes it handles and what happens when required content cannot be read. The source policy travels with the task so that a cached result from one policy cannot be mistaken for an answer produced under another.

4. Choosing what to represent

After parsing, the system needs a representation that fits the work. A learning guide might preserve a decision threshold, an exception, a mechanism, a visual comparison, or a sequence of steps. A policy brief may need obligations, owners, effective dates, and unresolved contradictions. A procedure manual may need ordered actions and stopping conditions. Treating each as a generic “concept” makes the data easy to route but can erase the distinction that matters to the reader.

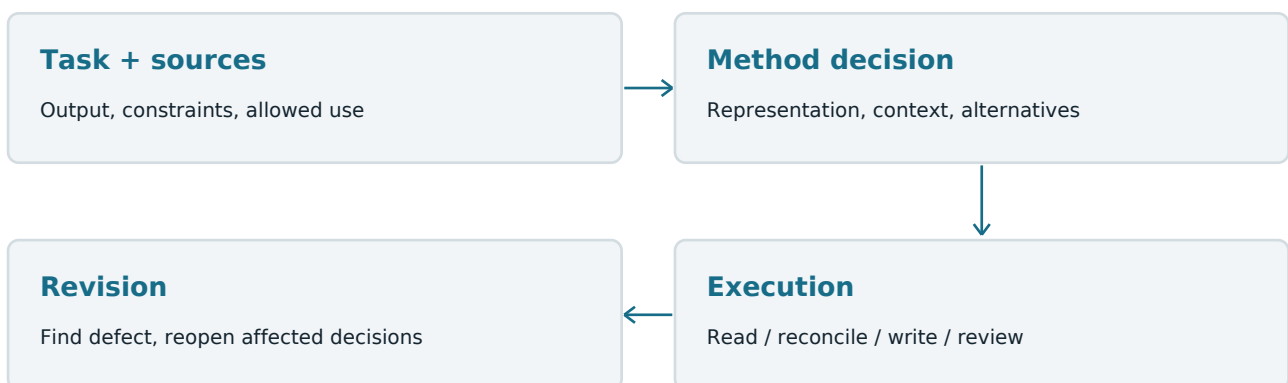
The proposed representation starts with source-bound candidate objects. Each object records its type, scope, source regions, qualifiers, and the task it supports. Relationships may include prerequisite, contrast, variant, conflict, and same target. The object type and relation set are chosen for the task; they are not a compulsory ontology for every document. A model can suggest

objects and connections, while validation keeps their source ownership visible. A human or second reader must examine consequential semantic decisions such as combining two apparently similar rules.

Selection follows representation. The system should record whether an object was chosen for the output, deferred to a deeper section, rejected as out of scope, or left unresolved. It should say why in terms of the task, not merely provide a relevance score. Some jobs demand exhaustive coverage; others benefit from a deliberately small route. A method that requires every extracted fact to appear in the final lesson would produce bloated material. A method that selects aggressively without an omission trail can quietly lose the point that motivated the request. The policy decides the acceptable balance.

Method selection is a separate decision. Candidate methods should carry applicability conditions, exclusions, source assumptions, expected outputs, and known failure modes. A selector could compare a source-order draft, a topic-board synthesis, and a decision-first route for a particular brief. It should explain which conditions fit and where the evidence is weak. The current method file stores applicability prose, but the operator chooses it manually. A future selector would need a test set showing that its choices improve accepted work on new tasks. Text similarity between a request and a method description is insufficient evidence.

5. A graph that expresses meaning and work



Proposed integrated architecture. The current execution paths remain separate.

Let a production run have jobs V . A hard-result edge $u \rightarrow v$ means v must receive u 's completed result. A shared-plan edge says two jobs use the same versioned route or board, even if neither waits for the other's final prose. An evidence-validity edge marks that a claim or artifact depends on a source region or semantic decision and may need review if it changes. Resource constraints limit how many jobs can use a model account, CPU parser, audio renderer, or human reviewer at once. These relationships do different things; putting them all into a single undifferentiated `depends_on` list would either over-serialize the work or miss a reason to reopen it.

The current method runner implements hard dependencies and named resource lanes. Ready nodes run within a global worker cap and each lane's capacity. A node receives only its selected task fields, its own instructions and input, and the completed results of the nodes it names. This prevents future-result leakage and allows a changed branch to leave an independent branch cached. The guide pipeline uses a fixed sequence: parallel unit extraction, one global reconciliation call, one global plan call, then parallel lesson authoring and per-lesson review. Its writers also see a

shared route and exact source evidence for prerequisite lessons. They do not wait for completed prerequisite prose.

The next architecture should keep several composition policies available. Parallel writers with a common route minimize the longest serial path. Writers that consume finished prerequisite prose may give better callbacks, but each required predecessor adds waiting and context. A cross-section editor could review parallel drafts and request local repairs instead of serializing all authors. A large reference collection may need hierarchical reconciliation, while a small collection may fit one global pass. These are choices to evaluate, not a single rule to hard-code into Lamina.

6. Context is assembled, not inherited

A worker needs enough context to do its assigned job and no more than the available model can use reliably. The packet may contain local source regions, nearby boundary material, a shared route, selected prior results, source conflicts, and previous reviewer observations. Each item has a reason to be there. A whole document pasted into every call can consume the budget, repeat irrelevant material, and make later changes invalidate far more work than necessary. A tiny excerpt can remove the condition that changes the answer. The useful packet depends on the job.

Define B_v as the input-token budget for job v after reserving space for its instructions and output. Let C_v be candidate context items with measured token sizes, and let P_v be protected items: for example the exact source span defining a safety condition or the current task constraint. Context assembly chooses a subset S_v subset of C_v with P_v subset of S_v and total size no greater than B_v . Ranking the remaining candidates by estimated relevance is an implementation choice, not a proof that the omitted material was unimportant. If protected material alone exceeds the budget, the system should split the job or stop for a scope decision. Silent truncation would break the task contract.

Context receipts should make the selection inspectable. They need the source and object IDs offered to the worker, versions of the shared plan, omitted candidate counts, and the actual request size. Exact consequential claims should remain traceable to reopened source regions. A compact global board can coordinate many writers, but it cannot substitute for the underlying source when an editor must decide whether two conditions are equivalent. The current guide pipeline's shared route and prerequisite evidence are a partial implementation. The generic method runner sends selected task fields and direct dependency results; its input object can carry more context, but it has no source-aware context assembler.

An operator can attach previously recorded observations to chosen method nodes. The runtime resolves only those IDs within the method family and includes them in input.experience. This is a real downstream consumer of stored experience. It also exposes the limit: the observation is evidence that someone recorded a judgment, not proof that the judgment applies to this task. A method that indiscriminately adds every past complaint to every worker would create a larger and less coherent prompt. Selection, applicability and effect need to be tested.

7. Parallelism, limits, and cost

Scheduling matters because a multi-document job has independent reading, global decisions, writing, review, and delivery. Let t_v be the measured service time of job v , $W = \sum_v t_v$ the total service work, and D the service time along the longest hard-dependency path. With p equivalent workers, elapsed time has the ideal lower bound $T \geq \max(W/p, D)$. If resource lane r has capacity c_r and the jobs using it demand W_r service time, then $T \geq \max(D, W/p, \max_r W_r/c_r)$ under the simplifying assumption that those lane demands are measured consistently. This is a bound, not an expected completion time. It excludes queue waits, provider throttling, retry attempts, setup, review, and delivery. Jobs with several resources at once need a richer schedule model.

The bound explains two engineering choices. Adding writers cannot remove a single long global planning call from the critical path. Forcing every writer to wait for the previous finished lesson may improve continuity while increasing D . The right comparison is therefore not the number of parallel agents or drafts per minute. It is time and total cost to an accepted artifact at the same requested scope and judged quality. A system can appear faster by producing less material, skipping difficult source regions, or shifting review work onto a person.

Context also has a cost. Report input-token amplification as total semantic input tokens divided by tokens in distinct accepted source text, with stage-level totals and the largest single request. That ratio is descriptive. Repeating a prerequisite may prevent a serious error; repeating an entire collection for every local writer may waste money. Real provider charges, local machine use, storage, and human review time should be recorded separately. If a provider does not report a charge, mark any computed equivalent as an estimate. Lamina's checked-in systems benchmark exercises cached SQLite jobs with declared sleeps. It measures scheduler shape and cache reuse, not model latency or artifact quality.

Queueing claims also need care. Document jobs often arrive in bursts; service times vary; calls may share account rate limits. A steady-state formula fitted to constant independent arrivals can be misleading for a finite dependency graph. The useful data are per-lane arrival and start times, service-time distributions, observed concurrency, throttling, retries, and the tail time to accepted output. Wider concurrency may improve one stage while increasing contention or review backlog elsewhere.

8. Findings should repair the cause

A review that says only pass or revise is enough to block a poor artifact. It is not enough to explain the cause. A missing paragraph may trace to parsing, extraction, selection, route planning, authoring, or export. A false statement may have a real quote attached to it. If a reviewer finds a wrong conclusion about a changed source, editing the final sentence alone leaves the upstream interpretation available for the next run.

The proposed review result is a typed finding attached to the exact artifact and source versions it examined. A finding names the affected object or claim, the observed defect, the supporting source region, the reviewer, and the decision needed. Example types include unreadable input, missing object, false merge, wrong priority, unsupported claim, unanswerable question, broken cross-section reference, and failed delivery. A human reviewer can mark a finding unresolved when the sources themselves disagree. The system should not turn a model's uncertainty into a

confident correction by default.

Repair scope depends on the finding. A typographic defect in one lesson may require re-authoring that lesson and re-exporting. A missing exception can require a new extraction, global selection or plan review, several lessons, and their assessment prompts. An evidence-validity edge identifies work that must be *considered* when a source or decision changes. It does not prove that every descendant must be regenerated. Conversely, unchanged summary text does not always prove an old review is still valid: a source page can change while a model returns the same sentence. Protected source fingerprints and reviewer verdicts must participate in the right cache identity.

For an observation o , report repair effort $R(o)$ as a tuple: service seconds for re-executed jobs, human review minutes, monetary charges, and replacement-delivery count. These quantities must remain separate unless an explicit pricing model converts them to a common unit. The transitive dependency closure is an upper set of candidates for recheck, not the value of $R(o)$. A smaller invalidation set is an improvement only if it preserves the correct artifact. The present method runner reuses nodes when their request and adapter identity are unchanged and reruns a descendant when a dependency result changes. The guide pipeline caches validated stage responses. Neither path currently implements typed findings and an automatic route from a defect to the responsible upstream decision.

9. Experience must change a later decision

A reusable method has two different kinds of persistence. The first is operational: a completed node can be reused when its semantic inputs have not changed. The second is methodological: evidence from one run may lead an operator to choose a different representation, context policy, reviewer, or work graph for a later task. A cache provides the first. It does not provide the second.

Lamina stores local run receipts and operator observations. The browser can attach a review note to a node from a completed run, insert its ID into an edited method, and submit a new run. A method node can consume observations explicitly named by ID. That is more than a passive notebook: the next adapter call receives the selected text as attributed reference data, and the selection affects cache identity. The runtime does not inspect the corpus of observations, decide which method applies, write a new method version, or verify that the advice improved the next output. A useful future selector would present candidate methods with their applicability conditions and counterexamples, then let an operator or agent choose with a recorded reason.

A revision should be a testable hypothesis. State what failure it is meant to fix, what steps or context it changes, which invariants it must preserve, and what it may cost. Give the old and new methods new source collections that neither version used during tuning. Compare acceptance, omissions, repair effort, latency, and cost; keep the cases where the revision loses. Only then promote a version for broader use. A single approving model call or an attractive case study is not evidence of transferable learning.

This distinction is relevant to buyers. A team may want repeatable, reviewable production without autonomous adaptation. Another may value recommendations drawn from prior runs. Those are different products with different trust requirements. Lamina currently supports explicit reuse and observation selection. Automatic method selection and tested adaptation remain research and engineering work.

10. Delivery is another dependent job

The produced file is part of the workflow, not an administrative afterthought. A delivery step should identify the artifact bytes, destination, time, access check, and the source and method versions that produced it. If a document changes, the prior delivery receipt tells a team which published version people may still be using. A review pass on a source-linked JSON bundle does not prove that the PDF rendered legibly or that a web reader works on the target device. A script is not an audio file, and an audio file is not a listened-to lesson.

The current guide path exports a static reader and Markdown or PDF. Its public example can be inspected without a model account. The browser's active run registry is in memory; committed stage results survive interruption, but the browser does not automatically resume a prior run after restart. The method runner writes final receipts and can reuse committed nodes on a rerun. Verified external publication, media QA, and observed reader use are outside the current release. A future delivery node should report its own failures without silently recasting a completed authoring stage as a delivered product.

11. Worked example: a changed timeout rule

Consider a fictional team maintaining a short guide for engineers who run document-export workers. Three source files arrive. The operations note says a worker's lease expires after eight minutes and another worker may then claim the job; the first worker might still be running. An older onboarding note says expiry guarantees that the first worker has stopped. The commit specification requires the finishing worker to present an owner token that matches the current job record before publishing an export. The customer wants a guide that explains what to do when an old worker finishes after a replacement has claimed the work.

The contradiction is a production decision, not a wording problem. A reader that extracts “lease expires after eight minutes” from each note and merges them by topic can miss the opposed claims about whether the old worker can still finish. A guide writer could cite the onboarding note exactly and still teach an unsafe rule. A source-aware method should keep the two claims separate, mark their conflict, show which file has current authority, and preserve the owner-token condition as a distinct decision object. If source authority is unknown, the conflict remains open for a reviewer rather than being resolved by model confidence.

An illustrative task contract names engineers as readers, a one-page guide plus one changed-situation practice prompt as outputs, the operations note and commit specification as current teaching sources, and the older onboarding note as historical conflict evidence. The method could read the three files in parallel, reconcile conflicts on a compact board, plan a two-part guide, write a rule explanation and a failure scenario concurrently, then run a cross-section check before export. The explanation writer needs the current lease rule and the token requirement. The scenario writer needs those objects plus the intended decision change. Neither needs an unrelated deployment appendix. The cross-section reviewer checks that the scenario's answer agrees with the guide, that the expired worker is not treated as terminated, and that the token check is not reduced to an informal heartbeat.

Now the operations note changes the timeout from eight minutes to five, and the onboarding note is corrected to say that expiry does not stop the old process. A source fingerprint makes the change visible. The lease-duration object and the formerly conflicting claim need review. The

commit-token rule may remain the same, although the final guide and any question mentioning eight minutes need revision. If the guide uses a shared plan that includes the timeout, its writers may need fresh context. If a writer receives completed prerequisite prose, that prose must be rechecked before reuse. The repair trace should show the changed source regions, affected objects, jobs that actually ran, cached jobs accepted after review, and the exact new PDF or page delivered.

This walkthrough is the **proposed integrated architecture**. Lamina has not executed that complete conflict-to-delivery sequence. Its current repository has a simpler source-linked guide pipeline and a separate deterministic method example with independent readers, a dependent writer, selective cache reuse, and an operator observation. Those fixtures demonstrate scheduling and structural validation. They do not demonstrate automatic conflict adjudication, typed finding repair, or a verified published revision.

12. How to evaluate the claim

The useful competitive question is narrower than whether Lamina has agents, audio, or a graph. Manus documents automatic parallel research and approved updates to project context [1][2]. Google's Notebook product documents source-based audio and generated study aids [3][4]. LangGraph documents graph execution, persistence, and human interruptions [5][6]. Lamina should not claim those basic capabilities are novel or absent elsewhere. Product documentation does not reveal every internal decision a competitor makes. The proposed advantage is a source-to-decision-to-repair trail that a team can use when the source or brief changes. It remains unproven until a comparison measures that task end to end.

The test should use a new, permissioned source collection with prose, a table or figure, overlap, and a deliberate conflict. Freeze the task brief, allowable source roles, target deliverables, adapter versions, cost budget, and acceptance rubric. Independent reviewers should identify important source objects and judge the finished output without knowing which workflow made it where possible. Compare the fixed guide path, an integrated method-driven alternative when built, a careful source-order baseline, and any external products under the same legal source and target-output conditions. A framework comparison requires an actual implemented workflow, with its build time stated separately.

Measure the first usable draft and the accepted, accessible artifact. Record missing consequential objects, wrong merges, unsupported statements, teaching or operational usability, reviewer effort, model tokens, actual charges where reported, elapsed time, retries, and local resource use. Then change one source and seed one review finding. Ask a fresh operator to locate the cause, repair the artifact, and identify what was republished. Measure the service work actually repeated, the work reused safely, and any new error introduced by the repair. Repeat across more than one corpus and report failures and tail times, not only the best demonstration.

The old and proposed systems may win on different dimensions. A plain source-order guide may be fastest for a small clean file. A global plan may give a clearer route for a compact collection. Bounded local readers and a shared board may help on a large heterogeneous set, but add synthesis and review cost. Completed-prerequisite writing may improve cross-section references while slowing production. These are hypotheses with identifiable measurements. The current synthetic benchmark supplies only scheduling and cache timings for declared sleeps. It cannot justify claims about model speed, output quality, or customer value.

If a future run shows that an unfamiliar operator can reuse a method on new material, catch a source-derived omission, repair the right decisions, and deliver a comparable or better artifact with less total effort, that would support a specific product claim. Until then, Lamina is an inspectable engineering base with one working document application and a separate reusable job runtime. The integration and its benefit remain to be demonstrated.

References

The product-capability links below were checked against official documentation on 21 September 2026. They support only the stated capability descriptions. The architecture and worked example in this paper are Lamina's design and do not derive from those product pages.

1. Manus, [“What is Wide Research?”](#).
2. Manus, [“Projects That Learn From Every Task”](#) and [Project Skills](#).
3. Google, [“Generate Audio Overview”](#).
4. Google, [“Generate Flashcards or Quizzes”](#).
5. LangChain, [“Graph API overview”](#) and [Persistence](#).
6. LangChain, [“Interrupts”](#).